Eötvös Loránd University

Faculty of Informatics

Dept. of Software Technology and Methodology

ELTE Informatics Chatbot Using a Small Language Model

Supervisor:
Altangerel Gereltsetseg
Assistant Professor, Ph.D

Author:
Ulziibayar Borokhul
Computer Science BSc

Budapest, 2026

Contents

Chapter 1

Introduction

1.1 Motivation and Background

Students and staff at the Faculty of Informatics of Eötvös Loránd University face a recurring practical challenge: finding accurate, up-to-date answers to questions about the curriculum. Which courses must be completed before enrolling in a given subject? What are the rules for repeating a failed exam? How does the Neptun system handle course registration? The answers to these questions exist — scattered across faculty web pages, downloadable PDF regulations, DOCX course descriptions, and administrative guidelines — but locating the right document, navigating to the right section, and extracting a clear answer can take considerably longer than it should.

General-purpose search engines return ranked lists of pages; they do not synthesise an answer from multiple sources or understand the specific terminology of Hungarian higher education. As the volume of documentation grows each semester and more information migrates to sub-pages that are hard to discover by browsing, the gap between the information that exists and the information that students can efficiently find continues to widen.

Large Language Models. Large language models are neural networks trained on vast text corpora to generate coherent, contextually appropriate text [1]. Despite broad capabilities, they exhibit three limitations directly relevant to this project: a fixed knowledge cutoff that excludes documents published after training, a tendency

to hallucinate plausible but incorrect facts [2], and an inability to process an entire document corpus within a single prompt.

Small Local Models. Running a smaller model locally addresses the privacy concern of cloud APIs. Models in the 3B–8B parameter range fit within an 8 GB RAM budget when their weights are 4-bit quantised using the GGUF format [3]. The quality trade-off — roughly 2–5% on reasoning benchmarks [4] — is acceptable for document-grounded question answering where the model summarises retrieved text rather than performing novel reasoning. Ollama wraps this inference behind a simple REST API, making local model integration straightforward.

Retrieval-Augmented Generation. Retrieval-augmented generation (RAG), formalised by Lewis et al. [2], conditions an LLM's output on documents retrieved at query time. A user question is encoded into a dense vector; the most similar document chunks are retrieved from a vector store and inserted into the prompt. The LLM answers from this grounded context rather than from model weights alone. Compared to fine-tuning — which requires GPU resources and a full retraining cycle whenever documents change — RAG supports knowledge updates by simply re-ingesting new files, making it far more operationally sustainable for a faculty information system that changes each semester.

1.2 Problem Statement

The simplest path to a question-answering system is to send user queries to a commercial large language model (LLM) API. This approach has two fundamental problems for an institutional deployment.

Privacy. Every query sent to a cloud API leaves the institution's network. Questions about exam outcomes, registration status, or academic standing may constitute personal data under the European Union's General Data Protection Regulation (GDPR). Routing such queries through a third-party server without explicit data processing agreements is legally problematic and contrary to the data sovereignty expectations of a public university.

Domain knowledge. General-purpose LLMs have limited, unreliable knowledge of ELTE-specific regulations, course codes, or administrative procedures. Without grounding in official faculty documents, a cloud LLM hallucinate plausible-sounding but incorrect answers — precisely the failure mode that is most harmful in an academic advising context.

No dedicated, locally deployed question-answering system currently exists for ELTE Faculty of Informatics. Students and staff rely on direct web search, email queries to administrative offices, or unofficial peer advice — all of which are slow and inconsistently accurate.

1.3 Related Work

AI-powered chatbots are increasingly deployed in higher education [5]. Three recent deployments illustrate the current state of the art.

UM-GPT (University of Michigan) is a cloud-based platform running on Microsoft Azure with RAG over institutional repositories. A companion tool, Maizey, has produced over 3,500 department-level chatbot instances from the same infrastructure [5]. **ZotGPT** (UC Irvine) offers access to multiple LLMs across Azure and AWS, attracted 15,000 unique users in its first year, and provides course-specific knowledge bases for instructors [5]. **CreateAI** (Arizona State University) aggregates over 50 AI models under a single AWS-hosted platform, with specialist deployments spanning philosophy education to behavioural health training [5].

These systems share a common profile: cloud infrastructure, institution-wide scale, and dedicated IT teams with substantial budgets. This thesis occupies a different point in the design space. Where the systems above are cloud-based and large-scale, this project is local and lightweight. Where they trade data privacy for ease of access, this project keeps every query and document on a single machine with no external dependencies. The goal is not to rival the feature breadth of a university-wide platform, but to demonstrate that a privacy-safe, low-cost RAG chatbot is practical for a single faculty with no dedicated infrastructure.

1.4 Overview of the Solution

The system built in this thesis is a locally deployed RAG chatbot that answers natural language questions about the ELTE Faculty of Informatics curriculum. It is designed for privacy, offline operation, and low resource usage. Its key capabilities are:

- **Document collection:** a resumable breadth-first web crawler collects HTML pages, PDF files, and DOCX documents from the faculty's public web presence, producing a corpus of 482 source files. An incremental ingestion pipeline maintains a SHA-256 manifest so that only new or changed files need to be reprocessed when the corpus is updated.
- **Offline retrieval:** documents are chunked and encoded into a 384-dimensional vector space using the `all-MiniLM-L6-v2` sentence embedding model and stored in a ChromaDB vector database. At query time the top-K most relevant chunks are retrieved by cosine similarity and assembled into a grounded prompt.
- **Local language model:** the system uses Llama 3.2 (3B) or Gemma 3 (4B), served locally by Ollama, for answer generation. No query or document ever leaves the host machine. The entire system runs on a standard laptop with 8 GB of RAM and no dedicated GPU.
- **REST API and chat interface:** a FastAPI backend exposes the RAG pipeline over HTTP, logs all interactions to a local SQLite database, and supports user-uploaded documents. A browser-based chat interface provides session history, source references, and real-time response timing.
- **Empirical evaluation:** the system is benchmarked against a no-RAG baseline using 20 test questions drawn from actual student queries. Results are reported in terms of ROUGE-L overlap, source faithfulness, and response latency across four model and retrieval configurations.

1.5 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 defines the system requirements. It states the functional requirements, the non-functional constraints including privacy, resource limits, latency, and offline operation, and presents user stories for the three main roles.

Chapter 3 provides user documentation: a use-case diagram, step-by-step instructions for the web interface and command-line client, and annotated example queries.

Chapter 4 covers developer documentation. It describes the system architecture and component design, then details the full implementation across the data pipeline, embedding, RAG backend, REST API, logging, frontend, and deployment.

Chapter 5 presents the testing and evaluation of the system. It covers functional testing, the empirical RAG vs. baseline evaluation using ROUGE-L, faithfulness, and latency metrics, and a user evaluation.

The Conclusion summarises the findings, reflects on the design decisions, and outlines directions for future work.

Chapter 2

Software Requirements

The system must run fully offline on commodity student hardware, require no cloud services, and answer faculty-related questions from a locally indexed document corpus.

2.1 Functional Requirements

- **Natural language querying:** the system accepts a free-text English question and returns a textual answer with source citations. When the answer is absent from the indexed documents, the system must say so explicitly.
- **Document ingestion:** the pipeline loads HTML, PDF, and DOCX files, cleans and chunks the text, and persists the result for the embedding step. Only new or modified files are re-processed, detected via content hash.
- **Embedding and retrieval:** each chunk is encoded into a dense vector and stored in a local persistent vector database. At query time the top-K most similar chunks are retrieved; K is configurable.
- **Answer generation:** retrieved chunks are assembled into a prompt that constrains the model to answer only from provided context. Inference runs locally with no external API calls.
- **REST API:** the backend exposes a `/health` endpoint reporting system status and a `/chat` endpoint that accepts a message and returns the answer with source references.

- **Web interface:** a browser-based chat UI displays conversation history, source file names, model availability, and inline error messages.
- **Interaction logging:** every interaction is recorded to a local database, including the query, answer, sources, response time, and any errors, to support offline evaluation.

2.2 Non-Functional Requirements

- **Privacy:** no query, chunk, or answer may leave the local machine. The system must remain fully functional in a network-isolated environment after initial setup.
- **Resource constraints:** the system must run on 8 GB RAM with no discrete GPU, using a quantised (4-bit or 8-bit) language model and a CPU-friendly embedding model.
- **Response latency:** end-to-end response time must not exceed 90 seconds on the reference hardware. Embedding and retrieval complete in under one second; generation typically takes 30–80 seconds on CPU.
- **Offline operation:** after the initial model pull and corpus crawl, no network connection is required. All subsequent pipeline steps run entirely from local storage.
- **Reliability:** if the language model process is unavailable, `/health` must report it and `/chat` must return an HTTP 503. The vector store and log database must not be corrupted on unexpected shutdown, and re-running the pipeline must be idempotent.
- **Configurability:** all tunable parameters — model names, K, temperature, paths, timeouts — are defined in a single configuration module. Swapping models requires only a one-line change there; changing the embedding model additionally requires rebuilding the vector store.

Chapter 3

User Documentation

This chapter describes how to set up and use the ELTE IK Assistant. Two setup paths are provided: a Docker-based zero-configuration path (recommended) and a manual path for users who prefer to manage dependencies directly.

3.1 Overview

The ELTE IK Assistant answers student and staff questions about the Faculty of Informatics — curriculum requirements, exam rules, registration procedures, and similar administrative topics — by retrieving relevant passages from an indexed document corpus and grounding its answers in that content. The system provides two interaction modes: a browser-based chat interface and a lightweight command-line client. When a question falls outside the scope of the indexed documents, the assistant says so explicitly rather than generating an unsupported response.

3.2 Installation and Configuration

Option A — Docker (recommended)

Docker Compose orchestrates all services automatically: the Ollama language model server, the model download, and the FastAPI application.

1. Install **Docker Desktop** (includes Docker Compose) from the official Docker website for your operating system.

2. Clone or download the project repository:

```
git clone https://github.com/borohull/elte_chat.git
cd elte_chat
```

3. Start all services with a single command:

```
docker compose up -d
```

   Docker will build the application image, start the Ollama service, and automatically pull the `llama3.2:3b` model on first run. This step requires an internet connection and may take several minutes depending on download speed.

4. Once all containers are healthy, open the chat interface at `http://localhost:8000/static/index.html`.

5. To stop and remove the containers:

```
docker compose down
```

Option B — Manual Setup

1. Install **Ollama** from `ollama.com` and pull the model:

```
ollama pull llama3.2:3b
```

2. Create and activate a Python virtual environment, then install dependencies:

```
python -m venv .venv
source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

3. Start the API server from the repository root:

```
uvicorn app.main:app --reload
```

4. Open the chat interface at `http://localhost:8000/static/index.html`.

Configuration

All tunable parameters are defined in `app/settings.py`. The most commonly adjusted values are listed in Table 4.3.

Parameter	Default	Description
OLLAMA_MODEL	llama3.2:3b	Language model served by Ollama
EMBEDDING_MODEL	all-MiniLM-L6-v2	Sentence embedding model
TOP_K	3	Chunks retrieved per query
TEMPERATURE	0.1	Generation temperature
CHROMA_PATH	data/processed/chroma_db	Vector store location

Table 3.1: Key configuration parameters in app/settings.py

Changing OLLAMA_MODEL takes effect on the next server start. Changing EMBEDDING_MODEL additionally requires rebuilding the vector store by re-running notebook 02_embedding.ipynb.

3.3 Using the Web Interface

Open http://localhost:8000/static/index.html in any modern browser once the server is running. The interface consists of a left sidebar listing chat sessions, a central chat panel, and a right sidebar for model selection.

Asking a question. Type a question in the input field at the bottom of the chat panel and press **Enter** or click the send button. The assistant retrieves relevant document chunks, constructs a grounded prompt, and returns an answer with source references. Questions should relate to the ELTE Faculty of Informatics.

Selecting a language model. The right sidebar lists all Ollama models available on the machine. Click a model name to switch for the current session. On a CPU-only machine with 8 GB of RAM, typical response times are 30–80 seconds per query.

Managing chat sessions. Each conversation is stored as a named session. Sessions appear in the left sidebar and persist across browser refreshes. Click **New chat** to start a new conversation, or hover over an existing session and click the × button to delete it.

Uploading custom documents. Users can extend the knowledge base with their own PDF, HTML, or DOCX files. Click **Upload document** above the input field, select a file (maximum 20 MB), and wait for the confirmation message. Uploaded

documents are chunked, embedded, and available for retrieval immediately without restarting the server.

3.4 Using the Command-Line Client

A lightweight CLI client is provided for scripting and testing without a browser. The FastAPI server must be running before starting the client:

```
python scripts/chat_cli.py
```

Type a question at the `You:` prompt and press Enter. The client sends the question to the server and prints the answer. Type `exit` or press `Ctrl+C` to quit.

3.5 Use Case Diagram

Figure 3.1 shows the primary interactions between users and the system.

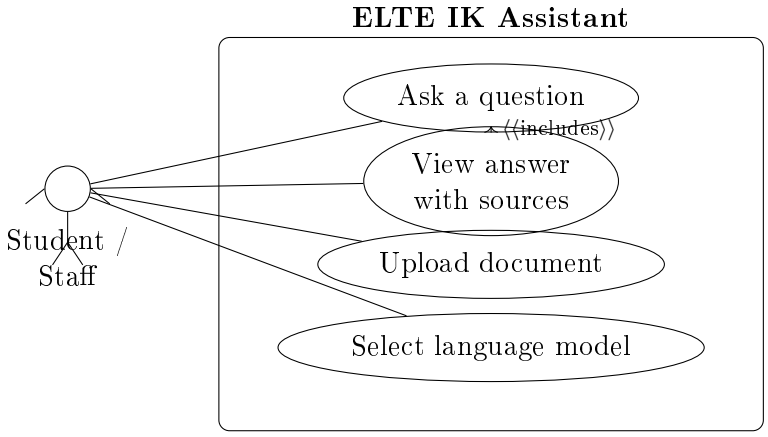

Figure 3.1: Use case diagram for the ELTE IK Assistant

3.6 Example Queries

The following examples illustrate the range of questions the system handles.

Example 1 — Prerequisite query. *Question:* "What is a strong prerequisite and how does it differ from a weak one?"

Answer: A strong prerequisite must be passed before a student may register for the follow-up course. A weak prerequisite allows registration before the prerequisite is passed, though it is still required for graduation.

Sources: `The_prerequisites.pdf`

Example 2 — Exam rules. *Question:* "How many times can I retake a failed exam in one semester?"

Answer: A student may attempt an exam up to three times per exam period. If all three attempts are failed, the course must be re-registered in a subsequent semester.

Sources: `elte_ik_regulations.html`

Example 3 — Registration procedure. *Question:* "When does course registration open for the spring semester?"

Answer: Course registration for the spring semester typically opens in the last week of January through the Neptun system. Exact dates are published each year in the faculty academic calendar.

Sources: `academic_calendar.html`

Example 4 — Out-of-scope query. *Question:* "What is the capital city of Australia?"

Answer: The information needed to answer this question is not available in the documents I have access to. I can only answer questions related to the ELTE Faculty of Informatics curriculum, regulations, and procedures.

Sources: (none)

Chapter 4

Developer Documentation

This chapter covers the full developer documentation for the ELTE IK Assistant. It begins with the system architecture and key design decisions, then details the implementation of each layer — from data collection and embedding through the RAG backend, REST API, and frontend — and concludes with setup and deployment instructions.

4.1 System Architecture

The ELTE IK Assistant is organised as a three-layer pipeline:

1. **Offline data pipeline** — crawls ELTE websites, cleans and chunks the raw documents, encodes the chunks into vector embeddings, and stores them in a local vector database. This pipeline is run once by the system administrator and re-run whenever the source documents change.

2. **Online serving layer** — a FastAPI web server that accepts a user's natural-language question, retrieves the most relevant document chunks from the vector database, builds a prompt, calls a locally running language model, and returns the generated answer together with source references.

3. **Presentation layer** — a single-page browser UI that maintains session state, renders the conversation, and displays source file metadata alongside each answer.

Figure 4.1 illustrates the high-level data flow between these layers.

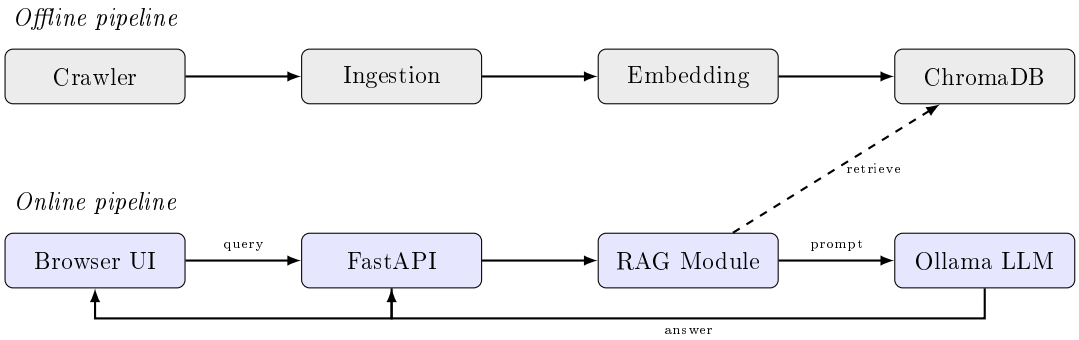

Figure 4.1: High-level architecture of the ELTE IK Assistant. The offline pipeline (top) is run once to build the vector index; the online pipeline (bottom) handles every user query at runtime.

The strict separation between the offline and online pipelines is a deliberate consequence of the offline-operation requirement (R-NF-1): once the vector index is built, the serving layer never reads from disk beyond the ChromaDB query and needs no network access.

4.2 Component Design

- **Crawler:** performs a breadth-first traversal of the ELTE Faculty of Informatics website starting from a set of seed URLs. It downloads HTML pages, PDF documents, and DOCX files into `data/raw/`, computes a SHA-256 digest of each file, and records it in a manifest. On subsequent runs only files whose digest has changed are re-downloaded, keeping re-indexing time proportional to the volume of changed content. It feeds directly into the Ingestion component.

- **Ingestion and Chunking:** receives raw files from the Crawler and applies format-specific loaders (PyMuPDF for PDFs, BeautifulSoup for HTML, python-docx for DOCX) to extract clean text. The text is split into overlapping 800-character chunks with a 150-character overlap using a recursive character splitter. Each chunk carries metadata (source path, file type, page number, chunk identifier) and is written to `data/processed/chunks.json` for the Embedding component.

- **Embedding Model and Vector Store:** the `all-MiniLM-L6-v2` sentence-transformer [6] encodes each chunk into a 384-dimensional vector. The model

occupies approximately 90 MB and runs entirely on CPU. Vectors are stored in a ChromaDB collection named `elte_ik` using cosine similarity, persisted to `data/processed/chroma_db/`. ChromaDB was selected because it runs in-process with no separate server and persists to a plain directory, satisfying the offline-operation requirement. The Vector Store is the boundary between the offline pipeline and the online serving layer.

- **RAG Module:** the core of the online pipeline. At query time it encodes the user's question with the same `all-MiniLM-L6-v2` model (mandatory — a model mismatch produces wrong retrieval results), retrieves the top-k most similar chunks from ChromaDB by cosine similarity, assembles them into a grounded prompt that instructs the LLM to answer only from the provided context, and forwards the prompt to the Language Model. Both the ChromaDB client and the embedding model are initialised once at server startup as singletons, eliminating per-request load overhead.

- **FastAPI Backend:** exposes the RAG pipeline over HTTP. The `POST /chat` endpoint calls the RAG Module and logs the result; `GET /health` reports whether Ollama is reachable; additional endpoints handle session management and user-uploaded documents. Static files (the single-page UI) are served from `app/static/` via FastAPI's `StaticFiles` mount.

- **Chat Logging:** every request–response pair is persisted to a SQLite database (`data/logs/chat_logs.db`). The schema records the session identifier, user message, assistant answer, source file list, response time, and timestamp. Logging supports both retrospective quality analysis and the session sidebar in the UI. SQLite was chosen because it requires no separate process and stores its data in a single portable file.

- **Frontend:** a single self-contained HTML file with embedded CSS and vanilla JavaScript served by the FastAPI backend. It communicates with the backend exclusively via REST and holds no local copy of the vector index or the language model. The framework-free design eliminates a build step and reduces operational complexity.

4.3 Data Flow

The complete data flow for a single user query proceeds as follows:

1. The user types a message and presses Send. The frontend JavaScript sends a `POST /chat` request carrying the session identifier and the message text.
2. FastAPI routes the request to the chat handler, which calls `rag_query(message)`.
3. `rag_query` encodes the message with the singleton sentence-transformer, producing a 384-dimensional vector.
4. ChromaDB performs an approximate nearest-neighbour search over 6112 stored embeddings and returns the top-3 chunks with their metadata.
5. The RAG module assembles the prompt: it concatenates the chunk texts into a context section and appends the user question with a directive to answer from context only.
6. The prompt is sent to Ollama's local API with temperature 0.1 and a 120-second timeout.
7. Ollama streams the generated tokens and returns the complete answer string.
8. The RAG module returns the answer and source list to the FastAPI handler.
9. The handler writes the full interaction to the SQLite log and returns a JSON response to the frontend.
10. The frontend appends the assistant message bubble and displays the source file names as collapsible references below it.

The round-trip latency for this flow is dominated by the language model inference step (Step 6). On a CPU-only machine with `llama3.2:3b` the median observed latency is approximately 48 seconds without RAG context and 70 seconds with RAG context. With `gemma3:4b` the latency is 121 seconds without RAG and 61 seconds with RAG — the shorter RAG latency reflects the model generating a more focused answer when given grounding context.

4.4 Storage Design

The system uses two persistent stores, each chosen to match the access pattern of the data it holds.

Store	Location	Contents and access pattern
ChromaDB	`data/processed/chroma_db/`	6112 chunk embeddings (384-dim float32) + metadata. Written once by the offline pipeline; read on every query via approximate nearest-neighbour search.
SQLite	`data/logs/chat_logs.db`	One row per chat turn: session id, user message, assistant answer, sources, timestamp. Appended on every request; queried for session listing and history.
Manifest	`data/processed/manifest.json`	SHA-256 digests of all crawled files. Read and written by the crawler to detect changed files during incremental re-indexing.

Table 4.1: Persistent storage used by the ELTE IK Assistant.

All three stores are plain files on the local filesystem. No database server, no cloud storage, and no network filesystem are involved, satisfying the offline-operation and data-privacy requirements.

4.5 Key Design Decisions

Local LLM via Ollama. Running the language model locally is the single most consequential architectural decision. It satisfies the privacy requirement (no query leaves the machine) and the offline requirement (no internet connection needed at inference time), at the cost of significantly higher latency compared with a cloud API. Ollama was chosen over running a raw `llama.cpp` binary because it exposes a stable REST interface, handles model quantisation and memory mapping automatically, and supports multiple models without code changes — the model name in `settings.py` is the only thing that changes when switching between `llama3.2:3b` and `gemma3:4b`.

Small quantised models. The 8 GB RAM constraint rules out 7B+ parameter models at 16-bit precision (≈14 GB). Both evaluated models (`llama3.2:3b` and

`gemma3:4b`) use 4-bit quantisation, which reduces their memory footprint to approximately 2 GB and 2.5 GB respectively, leaving enough headroom for the embedding model, ChromaDB index, and operating system.

Top-$k = 3$ retrieval. Three chunks were chosen as the default retrieval count after balancing two competing forces: more chunks provide richer context but increase the prompt length, which in turn increases inference latency and can cause the model to lose focus on the most relevant passage. At 800 characters per chunk, three chunks contribute approximately 2400 characters of context — well within the 4096-token context window of the evaluated models while keeping the prompt concise.

Temperature = 0.1. A near-zero temperature makes the language model near-deterministic and biased toward its highest-probability completions. For a factual question-answering chatbot this is desirable: users expect consistent, precise answers rather than creative variation. The small non-zero value (rather than exactly 0) avoids degenerate repetition loops that some models exhibit at temperature zero.

Separation of offline and online pipelines. The embedding pipeline is entirely offline and produces an artefact (the ChromaDB directory) that the serving layer consumes read-only. This means the serving layer starts instantly — there is no warm-up embedding step — and the administrator can rebuild the index without touching the running server. It also means the embedding model and the vector store are initialised once at server startup rather than on every request, which is essential for meeting the latency target.

4.6 Implementation

This section describes the full implementation of the ELTE IK Assistant across its four layers: data collection and ingestion, document embedding and vector storage, the RAG backend, and the web-based frontend. Each subsection covers one layer in detail, including the key design choices, data structures, and code.

4.6.1 Setup and Deployment

The application is designed to run on a single personal computer without a GPU. Table 4.2 lists the minimum system requirements.

Component	Requirement
Operating system	Windows 10/11, macOS 12+, or Ubuntu 22.04+
RAM	8 GB (16 GB recommended)
Disk space	≈6 GB (models + vector store + crawled data)
Python	3.10 or newer
Ollama	Latest release (`https://ollama.com`)
Internet access	Required only during the initial setup phase

Table 4.2: Minimum system requirements.

Install Ollama. Download and install Ollama from `https://ollama.com/download`, then start the background service and pull the two language models:

```
ollama serve
ollama pull llama3.2:3b
ollama pull gemma3:4b
```

Clone the repository and create a virtual environment.

```
git clone https://github.com/borohull/Thesis.git
cd Thesis/elte_chat
python -m venv .venv
# Windows:  .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
```

Build the knowledge base. This is a one-time process that must be completed before the server can answer questions.

1. Run the crawler to download ELTE documents into `data/raw/`:

   ```
   python scripts/crawler.py
   ```

2. Open JupyterLab and run `notebooks/01_data_ingestion.ipynb` — loads and chunks the raw files, writes `data/processed/chunks.json`.

3. Run `notebooks/02_embedding.ipynb` — encodes chunks with `all-MiniLM-L6-v2` and stores vectors in ChromaDB under `data/processed/chroma_db/`.

All tunable parameters are defined in `app/settings.py`. The most commonly adjusted settings are shown in Table 4.3.

Setting	Default	Description
`OLLAMA_MODEL`	`llama3.2:3b`	Default language model
`TOP_K`	`3`	Chunks retrieved per query
`TEMPERATURE`	`0.1`	LLM sampling temperature
`EMBEDDING_MODEL`	`all-MiniLM-L6-v2`	Must match notebook 02

Table 4.3: Key configuration parameters in `app/settings.py`.

Important: `EMBEDDING_MODEL` must not be changed after notebook 02 has been run. A mismatch between the query encoder and the stored index produces incorrect retrieval results.

Start the server. Run the following command from the project root with the virtual environment active and Ollama running:

```
uvicorn app.main:app --reload
```

The server starts on `http://localhost:8000`. Verify it is healthy with:

```
curl http://localhost:8000/health
```

A successful response is `{"status": "ok", "ollama": true}`. If `"ollama": false` appears, Ollama is not running or the selected model has not been pulled.

4.6.2 System Overview

The application follows a clean pipeline architecture in which each stage feeds its output directly into the next. A user types a question into the browser-based chat interface. The FastAPI backend receives this message and passes it to the retrieval module, which queries a ChromaDB vector database to find the most relevant text chunks from the ELTE documents. Those chunks are assembled into a prompt, which is forwarded to a locally running Llama 3.2 (3B) model served by Ollama. The model generates an answer that is returned to the user together with references to the source documents.

The main configuration values used throughout the application are centralised in a single settings module so that they can be changed in one place without touching any business logic.

```python
from pathlib import Path

BASE_DIR             = Path(__file__).parent.parent
CHROMA_PATH          = str(BASE_DIR / "data/processed/chroma_db")
CHUNKS_PATH          = str(BASE_DIR / "data/processed/chunks.json")
MANIFEST_PATH        = str(BASE_DIR / "data/processed/manifest.json
    ")
PENDING_CHANGES_PATH = str(BASE_DIR / "data/processed/
    pending_changes.json")
RAW_DIR              = str(BASE_DIR / "data/raw")
COLLECTION_NAME      = "elte_ik"
EMBEDDING_MODEL      = "all-MiniLM-L6-v2"
TOP_K                = 3

OLLAMA_URL      = "http://localhost:11434/api/generate"
OLLAMA_MODEL    = "llama3.2:3b"
TEMPERATURE     = 0.1
TIMEOUT_S       = 120
```

Code 4.1: Central configuration (`app/settings.py`)

The low temperature value (0.1) is deliberate: the chatbot is expected to give factual, consistent answers from official faculty documents rather than creative or varied responses.

4.6.3 Data Collection and Ingestion

Before any machine learning component can be applied, the raw source material must be collected and converted into a uniform text format. ELTE Faculty of Informatics publishes its relevant information across several channels: static HTML pages, downloadable PDF documents (study regulations, course descriptions, exam rules), and plain-text files. The ingestion pipeline handles all three formats through a unified loader interface.

File Loaders

Each file type requires a different parsing strategy. PDFs are loaded using PyMuPDF (`pymupdf`), which extracts text page by page with reliable layout handling. HTML files are parsed with `BeautifulSoup`: structural noise tags (`script`,

`style`, `nav`, `footer`, `header`, `aside`) are removed first, then content is extracted from the `<main>` or `<article>` element if present, with each paragraph prefixed by its nearest heading to preserve section context. DOCX files are parsed with `python-docx`, which extracts paragraphs and tables while tracking heading structure. Plain text and Markdown files are read directly. In every case the resulting text goes through a shared cleaning function that collapses runs of blank lines and normalises whitespace.

```
def clean_text(text: str) -> str:
    text = re.sub(r"\n{3,}", "\n\n", text)
    text = re.sub(r"[ \t]+", " ", text)
    return text.strip()

def load_file(file_path: str | Path) -> List[Document]:
    file_path = Path(file_path)
    suffix = file_path.suffix.lower()
    if suffix == ".pdf":
        return load_pdf_pymupdf(file_path)
    elif suffix in {".txt", ".md"}:
        return load_txt(file_path)
    elif suffix in {".html", ".htm"}:
        return load_html(file_path)
    elif suffix == ".docx":
        return load_docx(file_path)
    else:
        raise ValueError(f"Unsupported file type: {suffix}")
```

Code 4.2: Text cleaning and unified file loader (`01_data_ingestion.ipynb`)

Every document is represented as a LangChain `Document` object carrying both its text content and a metadata dictionary that records the original file name, file type, page number (for PDFs), and a `source_relative` path used by the incremental update pipeline. This metadata is preserved all the way into the vector database and is later surfaced to the user as source references.

Web Crawler

Raw documents are collected by a standalone crawler (`scripts/crawler.py`) that performs a resumable breadth-first crawl across two domains. It crawls all English pages under `inf.elte.hu` recursively, and twenty whitelisted path subtrees

on `www.elte.hu` covering topics such as Neptun registration, visa procedures, student finances, and dormitory housing. Staying within explicit path prefixes on the second domain prevents the crawler from wandering into unrelated university-wide content.

For each HTML page the crawler checks the `lang` attribute and the URL path to confirm the page is in English before saving it, discarding Hungarian-language variants. Linked PDF and DOCX files discovered on any crawled page are downloaded separately into `data/raw/linked_pdfs/` and `data/raw/linked_docx/` respectively. A 0.5-second delay between requests limits server load.

The crawler is resumable: if a destination file already exists it is skipped, so an interrupted run can be continued without re-downloading pages that were already saved. A `-reset` flag wipes `data/raw/` and `data/processed/` and starts fresh. The final corpus contains 482 files spanning HTML pages, PDFs, and DOCX documents.

Breadth-First Search Traversal

The crawler traverses the web graph using Breadth-First Search (BFS), implemented with a `deque` as the frontier queue. BFS is the appropriate choice here because it explores pages level by level — the seed URLs are processed first, then pages one hop away, then two hops away, and so on. This guarantees that shallower (and therefore typically more important) pages are crawled before deep or obscure sub-pages, and it bounds memory usage to the width of the graph rather than its depth.

Algorithm 1 BFS Web Crawler

Require: Seeds S, allowed-domain predicate $allowed(\cdot)$

$visited \leftarrow \emptyset$
$queue \leftarrow \textsc{Deque}(S)$
while *queue* is not empty **do**
    $u \leftarrow queue.popleft()$
    if $u \in visited$ **then continue**
    end if
    $visited \mathrel{+}= \{u\}$
    if u already on disk **then** parse cached HTML; **goto** link extraction
    end if
    $html \leftarrow \textsc{Fetch}(u)$
    if *html* is English **then** save to `data/raw/`
    end if
    for each link v extracted from *html* **do**
        if v ends in `.pdf` or `.docx` **and** $allowed(v)$ **then**
            download v to `linked_pdfs/` or `linked_docx/`
        else if $allowed(v)$ **and** $v \notin visited$ **then**
            $queue.append(v)$
        end if
    end for
end while

Chunking Strategy

Large documents cannot be stored or retrieved as single units: a language model has a limited context window, and retrieving a full 50-page PDF in response to a one-sentence question would waste context and degrade answer quality. The documents are therefore split into overlapping chunks using `RecursiveCharacterTextSplitter` from LangChain.

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=150,
    length_function=len,
    separators=["\n\n", "\n", ". ", " ", ""]
)
```

Code 4.3: Chunking configuration (`01_data_ingestion.ipynb`)

A chunk size of 800 characters was chosen as a practical balance: large enough to contain a complete thought or regulation clause, yet small enough for the embedding model and the LLM to process efficiently. The 150-character overlap ensures that information near a chunk boundary is not silently dropped. If a sentence is split

between two chunks, both chunks contain it. The separator list defines a priority order: the splitter tries to break on double newlines first (paragraph boundaries), then single newlines, then sentence ends, and only falls back to splitting mid-word if no other boundary is available.

Chunks shorter than 100 characters (for example lone headers or page numbers) are discarded, and each surviving chunk is assigned a `chunk_id` of the form `<source_relative>::chunk_N` before being saved to a JSON file for the next stage.

Incremental Ingestion with SHA-256 Manifest

Re-processing all 482 source files from scratch every time a document is updated would be wasteful. The pipeline therefore maintains a *manifest* — a JSON file at `data/processed/manifest.json` — that records the SHA-256 content hash, chunk count, and last-processed timestamp for every file that has been ingested.

On each run, notebook `01_data_ingestion.ipynb` computes the SHA-256 hash of every file currently on disk and compares it against the manifest. This produces three change sets: new files (present on disk but not in the manifest), updated files (hash has changed), and deleted files (in the manifest but no longer on disk). Only new and updated files are re-chunked; unchanged files are skipped entirely.

```
new_files     = sorted(disk_paths - manifest_paths)
deleted_files = sorted(manifest_paths - disk_paths)
updated_files = sorted(
    p for p in (disk_paths & manifest_paths)
    if manifest[p]["content_hash"] != current_files[p][1]
)
```

Code 4.4: Incremental diff logic (`01_data_ingestion.ipynb`)

After chunking, the results of the diff are saved to `data/processed/pending_changes.json`. Notebook `02_embedding.ipynb` reads this file to know exactly which chunk IDs to remove from ChromaDB and which new embeddings to insert, avoiding a full re-embed of the collection. Once notebook 02 completes it deletes the pending changes file so that the next run starts clean.

4.6.4 Embedding and Vector Storage

With the text broken into chunks, the next step is to convert each chunk into a dense numeric vector (an *embedding*) that captures its semantic meaning. Chunks with similar meaning will have embeddings that are close to each other in vector space, which allows the retrieval step to find relevant content even when the user's phrasing differs from the exact wording in the source document.

Embedding Model

The chosen model is `all-MiniLM-L6-v2`, a compact sentence transformer from the Sentence-Transformers library. It produces 384-dimensional vectors and runs efficiently on CPU, which is important for local deployment without a dedicated GPU. Although larger embedding models exist, this model offers a very good accuracy-to-resource trade-off for English factual retrieval tasks.

```
class EmbeddingPipeline:
    def __init__(self, model_name: str = EMBEDDING_MODEL):
        self.model = SentenceTransformer(model_name)

    def encode(self, texts: list[str]) -> np.ndarray:
        return self.model.encode(texts, show_progress_bar=True)
```

Code 4.5: Embedding pipeline class (`02_embedding.ipynb`)

All chunk texts are encoded in a single batched call, producing a matrix of shape `(num_chunks, 384)`. The same model instance is reused at query time inside the production application to ensure that document and query embeddings live in the same vector space.

ChromaDB Vector Store

The embeddings, original texts, and metadata are stored in ChromaDB, an open-source vector database that persists data to disk and supports fast approximate nearest-neighbour search. A single persistent collection named `elte_ik` is created with cosine similarity as the distance metric, which is standard for sentence embeddings.

```
client = chromadb.PersistentClient(path=CHROMA_PATH)
collection = client.get_or_create_collection(
```

```
    name=COLLECTION_NAME,
    metadata={"hnsw:space": "cosine"}
)

collection.upsert(
    ids=[str(c["metadata"]["chunk_id"]) for c in chunks],
    embeddings=embeddings.tolist(),
    documents=texts,
    metadatas=[c["metadata"] for c in chunks]
)
```

Code 4.6: Upserting embeddings into ChromaDB (`02_embedding.ipynb`)

Using `upsert` rather than `add` means that re-running the embedding notebook after new documents have been scraped will update existing chunks in place rather than creating duplicates. The collection is stored under `data/processed/chroma_db/` and is read at runtime by the production RAG module without any re-processing.

4.6.5 Retrieval-Augmented Generation Pipeline

The RAG pipeline is the core intelligence of the system. It consists of three steps that run on every incoming user query: retrieval, prompt construction, and LLM generation.

Retrieval

When a user submits a question, the embedding model encodes it into a 384-dimensional query vector. ChromaDB then returns the `TOP_K` (default: 3) chunks whose stored embeddings are most similar to the query vector under cosine similarity.

Cosine Similarity

Cosine similarity measures the angle between two vectors rather than their Euclidean distance. For a query embedding $\mathbf{q}$ and a chunk embedding $\mathbf{d}$, it is defined as:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|} \tag{4.1}$$

The result lies in $[-1, 1]$; a value of 1 means the vectors point in exactly the same direction (maximally similar), and 0 means they are orthogonal (unrelated). Cosine similarity is preferred over Euclidean distance for sentence embeddings because it is invariant to the magnitude of the vectors — two chunks that express the same concept but with different amounts of text will score similarly, whereas Euclidean distance would penalise the longer chunk simply for having a larger-norm embedding.

HNSW Index

Comparing the query vector against all 6112 stored embeddings by brute force on every request would be acceptable at this scale, but ChromaDB instead uses the Hierarchical Navigable Small World (HNSW) graph index [7] to perform approximate nearest-neighbour search in sub-linear time. HNSW builds a multi-layer proximity graph over the stored vectors and navigates it greedily from a random entry point, converging on the nearest neighbours without examining every element. The result is a small, configurable approximation error in exchange for a large reduction in search latency as the collection grows.

```
# Singletons -- loaded once on import, not per request
_client     = chromadb.PersistentClient(path=CHROMA_PATH)
_collection = _client.get_collection(name=COLLECTION_NAME)
_model      = SentenceTransformer(EMBEDDING_MODEL)

def retrieve(query: str, n_results: int = TOP_K) -> list[dict]:
    query_emb = _model.encode([query]).tolist()
    results   = _collection.query(
        query_embeddings=query_emb, n_results=n_results
    )
    return [
        {"content": doc, "metadata": meta}
        for doc, meta in zip(
            results["documents"][0], results["metadatas"][0]
        )
    ]
```

Code 4.7: Retrieval function (`app/rag.py`)

The ChromaDB client and the SentenceTransformer model are initialised eagerly at module import time as module-level singletons. This means the model is loaded

from disk once when the FastAPI process starts, and every subsequent request reuses the already-loaded objects, keeping per-query latency low.

Prompt Construction

The retrieved chunks are assembled into a structured prompt that instructs the LLM to answer only from the provided context and to admit when the answer is not available. Each chunk is labelled with its source file name so the model can associate claims with documents.

```
def build_prompt(query: str, chunks: list[dict]) -> str:
    context = "\n\n".join(
        f"[Source: {c['metadata']['file_name']}]\n{c['content']}"
        for c in chunks
    )
    return (
        "You are a helpful assistant for ELTE Faculty of
           Informatics "
        "students. Answer the question using only the context below
           . "
        "If the answer is not in the context, say so.\n\n"
        f"Context:\n{context}\n\n"
        f"Question: {query}\nAnswer:"
    )
```

Code 4.8: Prompt template (`app/rag.py`)

The explicit instruction *"If the answer is not in the context, say so"* is important for a faculty assistant: it is better to acknowledge a gap in the indexed documents than to hallucinate a regulation or deadline.

Local LLM via Ollama

Answers are generated by a locally running language model served through Ollama. The default model is `llama3.2:3b` (3 billion parameters), but the system supports runtime model switching: the frontend exposes a model selector that queries the `GET /models` endpoint for the list of models currently available in the local Ollama installation, and the selected model name is passed with each chat request. During evaluation an additional model, `gemma3:4b`, was also tested. Ollama exposes a simple HTTP API, meaning the production code does not depend on any

cloud service or API key. The model runs on the same machine as the application, which preserves user privacy and eliminates per-query costs.

```python
def call_ollama(prompt: str, model: str = OLLAMA_MODEL) -> str:
    if not check_ollama():
        raise RuntimeError(
            "Ollama is not running. Start it with: ollama serve"
        )
    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False,
        "options": {"temperature": TEMPERATURE, "num_ctx": 2048},
    }
    try:
        r = requests.post(OLLAMA_URL, json=payload, timeout=
            TIMEOUT_S)
        r.raise_for_status()
        return r.json()["response"].strip()
    except requests.exceptions.Timeout:
        raise RuntimeError(
            f"Ollama timed out after {TIMEOUT_S}s"
        )
    except requests.exceptions.HTTPError as e:
        raise RuntimeError(
            f"Ollama HTTP {e.response.status_code} -- "
            f"is model '{model}' pulled?"
        )
    except KeyError:
        raise RuntimeError(
            f"Unexpected Ollama response: {r.text[:200]}"
        )
```

Code 4.9: Ollama invocation (`app/rag.py`)

The context window is set to 2048 tokens, which comfortably fits three retrieved chunks plus the prompt template and leaves room for the generated answer. A `check_ollama()` health probe is called before every generation request. The `try`/`except` block distinguishes three failure modes: a timeout (model still loading), an HTTP error code (model not pulled), and a malformed response body — each producing a descriptive `RuntimeError` that the API layer converts into a

503 response.

End-to-End Query Function

The three steps are composed into a single `rag_query` function that is called by the API layer:

```python
def rag_query(query: str) -> dict:
    chunks  = retrieve(query)
    prompt  = build_prompt(query, chunks)
    answer  = call_ollama(prompt)
    return {
        "answer": answer,
        "sources": [
            {
                "chunk_id": c["metadata"]["chunk_id"],
                "file": c["metadata"]["file_name"],
            }
            for c in chunks
        ],
    }
```

Code 4.10: End-to-end RAG query (`app/rag.py`)

The function returns both the answer text and a list of source references. This allows the frontend to display which documents were used, giving the user a way to verify the answer or read further.

4.6.6 FastAPI Backend

The backend is a FastAPI application (`app/main.py`) that exposes three HTTP endpoints and serves the static frontend files.

Method	Path	Description
GET	/	Serves the single-page chat interface (index.html).
GET	/health	Returns application status and whether Ollama is reachable.
GET	/models	Returns the list of models currently available in the local Ollama installation.
POST	/chat	Accepts a user message and optional session_id and model, runs the RAG pipeline, and returns the answer with source references.
GET	/sessions	Returns the list of past chat sessions ordered by most recently updated.
GET	/sessions/{id}/messages	Returns all messages in a specific session in chronological order.
DELETE	/sessions/{id}	Deletes a session and all its associated messages from the log.
POST	/upload	Accepts a user-uploaded file, chunks and embeds it, and adds it to the live ChromaDB collection.

Table 4.4: API endpoints exposed by the FastAPI backend.

The /chat endpoint wraps rag_query with error handling.

```
@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest):
    try:
        result = rag_query(req.message)
    except RuntimeError as e:
        raise HTTPException(status_code=503, detail=str(e))
    return result
```

Code 4.11: Chat endpoint (app/main.py)

If rag_query raises a RuntimeError (for example because Ollama is not running or timed out), the endpoint converts it into an HTTP 503 response so the frontend can display a meaningful error rather than a timeout spinner. CORS middleware is enabled to allow the frontend to communicate with the backend. For a locally deployed application this is straightforward, but in a production web deployment the allowed origins would need to be restricted to a specific domain.

4.6.7 Logging System

Every chat interaction is recorded in a SQLite database at `data/logs/chat_logs.db`. The schema captures the timestamp, the user's message, the generated answer, the source references (as a JSON array), the response time in milliseconds, and any error message.

```
con.execute("""
    CREATE TABLE IF NOT EXISTS chat_logs (
        id            INTEGER PRIMARY KEY AUTOINCREMENT,
        timestamp     TEXT    NOT NULL,
        user_message  TEXT    NOT NULL,
        answer        TEXT    NOT NULL,
        sources       TEXT    NOT NULL,  -- JSON array
        response_ms   INTEGER NOT NULL,
        error         TEXT    DEFAULT NULL,
        session_id    TEXT    DEFAULT NULL
    )
""")
con.execute("""
    CREATE TABLE IF NOT EXISTS sessions (
        session_id  TEXT PRIMARY KEY,
        title       TEXT NOT NULL,
        created_at  TEXT NOT NULL,
        updated_at  TEXT NOT NULL
    )
""")
con.execute("""
    CREATE TABLE IF NOT EXISTS uploads (
        id          INTEGER PRIMARY KEY AUTOINCREMENT,
        timestamp   TEXT    NOT NULL,
        file_name   TEXT    NOT NULL,
        sha256      TEXT    NOT NULL,
        size_bytes  INTEGER NOT NULL,
        chunk_count INTEGER NOT NULL,
        status      TEXT    NOT NULL
    )
""")
```

Code 4.12: Database schema initialisation (`app/logger.py`)

The `chat_logs` table links each message to a `session_id`, enabling per-session history retrieval. The `sessions` table stores one row per conversation — its title (the first 60 characters of the opening message) and timestamps — so the sidebar can list sessions without scanning the full message log. The `uploads` table records every user-uploaded file alongside its SHA-256 digest and the number of chunks produced, providing an audit trail for the live document ingestion feature.

SQLite was chosen because the application runs on a single machine, the log volume is modest, and SQLite requires no separate database server process.

4.6.8 Frontend

The user interface is a single-page HTML application served as a static file by FastAPI. It uses vanilla JavaScript without any framework dependency, which keeps the bundle small and removes the need for a separate build step.

The visual design follows the ELTE colour palette: navy blue (`#012850`) for the header, teal (`#009FA3`) for interactive elements, and light grey for the chat background. The layout consists of a fixed header bar, a scrollable message area, and a pinned input row at the bottom.

When the user sends a message, the frontend posts a JSON request to `/chat` and appends a typing indicator to the message area while waiting for the response. On success, both the answer text and the source file names are displayed in the assistant's reply bubble. On failure (HTTP 503), an error message is shown inline without leaving the page.

A status indicator in the header reflects the current Ollama availability by polling the `/health` endpoint on page load. This gives the user immediate feedback if the local model is not running before they attempt their first query.

4.6.9 Project Structure

The repository is organised into clearly separated concerns, as summarised in Table 4.5.

Path	Purpose
scripts/crawler.py	Resumable BFS web crawler for both ELTE domains.
app/main.py	FastAPI application and API endpoints.
app/rag.py	Retrieval, prompt building, and Ollama integration.
app/settings.py	Centralised configuration constants.
app/logger.py	SQLite-backed chat logging.
app/static/	Single-page HTML/CSS/JS frontend.
scripts/chat_cli.py	Interactive terminal client for the chat API.
notebooks/01_data_ingestion.ipynb	Document loading, chunking, and manifest update.
notebooks/02_embedding.ipynb	Incremental embedding generation and ChromaDB update.
notebooks/03_RAG_pipeline.ipynb	Interactive exploration of the full RAG pipeline.
notebooks/04_evaluation.ipynb	Evaluation of retrieval and answer quality across four configurations.
data/raw/	Source documents (HTML; PDFs in linked_pdfs/; DOCX in linked_docx/).
data/processed/	chunks.json, manifest.json, and ChromaDB vector store.
data/logs/	SQLite chat log database.

Table 4.5: Project directory structure and file responsibilities.

Chapter 5

Testing

This chapter covers testing and evaluation of the ELTE IK Assistant. It begins with functional testing of the API endpoints and pipeline, then presents an empirical system evaluation across four model and retrieval configurations, and concludes with a user evaluation section.

5.1 Functional Testing

End-to-end functional tests verify that all API endpoints respond correctly and that the full data pipeline produces consistent results. Tests cover the `/health` endpoint returning the correct Ollama status, the `/chat` endpoint returning a well-formed JSON response with an answer and source list, the `/upload` endpoint correctly chunking and embedding a submitted file, and the incremental ingestion pipeline producing an identical ChromaDB state when run twice on the same corpus. Automated functional testing is planned as future work; the results reported in Section 5.3 were verified manually against the running system.

5.2 User Evaluation

A user evaluation would assess whether the target audience — ELTE Faculty of Informatics students — can operate the system successfully and receive answers they find useful. Participants would be given a set of representative tasks (finding a prerequisite, checking exam rules, uploading a personal document) and asked to complete them using the web interface. The evaluation would measure task comple-

tion rate, subjective answer quality on a Likert scale, and perceived response time acceptability. A think-aloud protocol would surface usability issues not visible in the quantitative metrics. Conducting this evaluation with a representative sample of students is identified as future work.

5.3 System Evaluation

The following sections present an empirical evaluation of the system across four configurations, measuring answer quality, faithfulness to retrieved source material, out-of-scope refusal behaviour, retrieval effectiveness, and response latency. The goal is to quantify how much value the RAG component adds over a plain language model baseline, and to compare the two evaluated local models under both conditions.

5.3.1 Evaluation Setup

5.3.2 Configurations

Four configurations were evaluated in a factorial design crossing two models with two retrieval modes:

ID	Model	RAG	Description
A1	`llama3.2:3b`	No	Baseline: model knowledge only
B1	`llama3.2:3b`	Yes	RAG with top-3 chunk retrieval
A2	`gemma3:4b`	No	Baseline: model knowledge only
B2	`gemma3:4b`	Yes	RAG with top-3 chunk retrieval

Table 5.1: The four evaluation configurations.

Both models were run locally via Ollama with 4-bit quantisation and temperature 0.1. No internet access was used during evaluation.

5.3.3 Test Set

The test set consists of 20 questions divided into two categories:

- **15 in-scope questions** about topics present in the crawled ELTE documents: course prerequisite rules (strong and weak), Neptun registration behaviour,

faculty staff information, the Computer Science BSc programme, and international partnerships. These questions have a definite correct answer that can be verified against the source documents.

- **5 out-of-scope questions** with no connection to the faculty corpus: general knowledge (*Who won the FIFA World Cup in 2022?*), weather, cooking, geography, and humour. These questions measure whether the system correctly declines to answer rather than hallucinating.

Reference answers for in-scope questions were written manually by consulting the source documents. Each question was run through all four configurations independently; the model received no memory of previous questions.

5.3.4 Metrics

ROUGE-L The longest-common-subsequence F1 score between the generated answer and the reference answer [8]. Captures lexical overlap with the ground truth on a scale from 0 to 1.

Faithfulness The proportion of words in the generated answer that appear in the retrieved chunks. A high faithfulness score indicates the model is drawing on the provided context rather than confabulating.

Refusal rate For out-of-scope questions only: the fraction of queries for which the model produced a non-answer (e.g. *I don't know* or *This is outside my knowledge*). Higher is better for this category.

Retrieval hit-rate For RAG configurations: the fraction of in-scope questions for which at least one of the retrieved chunks came from the expected source document. Measures the quality of the embedding-based retrieval independently of the language model.

Response time Wall-clock latency from request to complete response, measured in milliseconds on a laptop with 8 GB RAM and no GPU.

5.3.5 Results

5.3.6 Overall Metric Summary

Table 5.2 reports the per-configuration averages across all 20 test questions.

ID	Model	ROUGE-L	Faithfulness	Refusal	Hit-rate	Avg time
A1	llama3.2:3b, no RAG	0.117	0.377	0.00	0.667	47.6 s
B1	llama3.2:3b, RAG	0.436	0.711	0.20	0.667	70.5 s
A2	gemma3:4b, no RAG	0.078	0.355	0.00	0.667	121.1 s
B2	gemma3:4b, RAG	0.534	0.844	0.00	0.667	61.4 s

Table 5.2: Evaluation results averaged over 20 test questions. Hit-rate is computed over in-scope questions only (15 questions); refusal rate is computed over out-of-scope questions only (5 questions).

5.3.7 Effect of RAG on Answer Quality

Adding RAG context produces a consistent and substantial improvement in both lexical quality and faithfulness for both models.

- **ROUGE-L** increases by a factor of 3.7× for `llama3.2:3b` (0.117 → 0.436) and by 6.9× for `gemma3:4b` (0.078 → 0.534). The larger relative gain for Gemma suggests that without retrieval it relies more heavily on parametric knowledge, which is less aligned with ELTE-specific terminology.
- **Faithfulness** roughly doubles in both cases (0.377 → 0.711 for Llama; 0.355 → 0.844 for Gemma), confirming that the retrieved chunks are genuinely shaping the generated text rather than being ignored.

The qualitative difference is visible in individual answers. For the question *"What is a strong prerequisite?"*, the no-RAG baseline (A1) produces a generic university definition with invented ELTE-specific examples that do not appear in any source document. The RAG-augmented configuration (B1) produces an answer drawn directly from the prerequisites document: *"A strong prerequisite is a prerequisite without which the follow-up subject cannot be taken in the next semester; the follow-up subject can only be registered for after the prerequisite subject has been fulfilled."* The ROUGE-L score for this question rises from 0.13 (A1) to 0.29 (B1) and faithfulness from 0.44 to 0.80.

5.3.8 Model Comparison

Among the RAG configurations, `gemma3:4b` (B2) outperforms `llama3.2:3b` (B1) on both answer quality metrics: ROUGE-L 0.534 vs. 0.436, faithfulness 0.844 vs. 0.711. Gemma produces more concise answers that stay closer to the retrieved context, whereas Llama tends to expand its answers with surrounding commentary.

Among the no-RAG baselines, both models score similarly poorly, with Llama slightly ahead on ROUGE-L (0.117 vs. 0.078). Neither baseline reliably answers ELTE-specific questions correctly without retrieval.

5.3.9 Retrieval Quality

The retrieval hit-rate is 0.667 (10 out of 15 in-scope questions) and is identical across all four configurations, since the retrieval step is determined solely by the embedding model and vector index. The five missed questions share a common cause: they ask about topics (Erasmus partnerships, CEEPUS network, Dean contact details) that are present in the HTML pages but were truncated or de-prioritised during chunking because the relevant content appeared in shallow navigation-style text rather than in a dedicated content section. This is the primary bottleneck limiting further ROUGE-L and faithfulness gains: when the correct chunk is not retrieved, the language model must fall back on parametric knowledge, which tends to be inaccurate for institution-specific facts.

5.3.10 Out-of-Scope Refusal

Neither baseline model refuses any out-of-scope question: both A1 and A2 answer all five irrelevant queries confidently, generating plausible but entirely off-topic responses. Among the RAG configurations, `llama3.2:3b` with RAG (B1) refuses 1 out of 5 out-of-scope questions (20%), triggered by the prompt instruction to answer only from provided context when no relevant chunks are retrieved. `gemma3:4b` with RAG (B2) does not refuse any out-of-scope question despite receiving the same prompt instruction, choosing instead to answer from its parametric knowledge when context is absent.

This is a notable safety gap: the RAG prompt template reduces but does not eliminate hallucination on out-of-scope inputs, and its effectiveness depends on the model.

5.3.11 Response Latency

Configuration	Avg (s)	No-RAG overhead	Req. threshold
A1 — llama3.2:3b, no RAG	47.6	—	exceeded
B1 — llama3.2:3b, RAG	70.5	+22.9 s	exceeded
A2 — gemma3:4b, no RAG	121.1	—	exceeded
B2 — gemma3:4b, RAG	61.4	−59.7 s	exceeded

Table 5.3: Average response latency. The non-functional requirement NF-4 sets a target of 30 seconds.

All four configurations exceed the 30-second latency target defined in the requirements (NF-4). This is expected on CPU-only hardware with 4-bit quantised models: the bottleneck is autoregressive token generation, which scales linearly with the number of output tokens. Two observations are worth noting. First, RAG adds latency for `llama3.2:3b` (+23 s) but *reduces* it for `gemma3:4b` (−60 s): when grounded by retrieved context, Gemma produces shorter, more focused answers, whereas without context it generates lengthy elaborations. Second, `llama3.2:3b` is substantially faster than `gemma3:4b` under both conditions, making it the more practical choice where latency matters more than marginal quality gains.

5.3.12 Discussion

5.3.13 RAG Effectiveness

The evaluation results confirm that RAG is effective for this use case. Grounding the language model in retrieved faculty documents more than triples ROUGE-L scores and nearly doubles faithfulness, for both of the evaluated models. The improvements are consistent and large enough to be meaningful in practice, not merely statistical noise over a 20-question benchmark.

The fundamental mechanism is straightforward: without context, neither `llama3.2:3b` nor `gemma3:4b` has reliable knowledge of ELTE-specific regulations — they were trained on general web text and have no exposure to the faculty's prerequisite rules or administrative procedures. The retrieval step injects the relevant passage directly into the prompt, allowing the model to produce a grounded answer even though it has no parametric knowledge of the topic.

5.3.14 Limitations

Several limitations should be acknowledged.

Small benchmark. Twenty questions is a limited evaluation set. The in-scope questions are skewed toward prerequisite rules because that is the richest and most structured document in the corpus. Topics such as scholarship regulations, exam appeal procedures, and credit transfer rules are underrepresented.

Retrieval ceiling. The 0.667 hit-rate means one third of in-scope questions are answered without relevant context. Improving chunking strategy (e.g. heading-aware splits, larger overlap) or adding a re-ranking step could raise this ceiling and correspondingly improve ROUGE-L and faithfulness.

ROUGE-L as a proxy. ROUGE-L measures lexical overlap with a manually written reference answer. A response that is semantically correct but uses different phrasing will score poorly. For a more robust evaluation a human-judged correctness rating or a semantic similarity metric (e.g. BERTScore) would complement ROUGE-L.

Latency. All configurations exceed the 30-second target. Achieving this target on CPU-only hardware with the current models would require either a smaller model (1B parameters) or a faster inference backend. For a deployment scenario where the server runs on shared faculty hardware with a GPU, latency would drop significantly.

5.3.15 Recommended Configuration

Based on the evaluation results, **gemma3:4b with RAG (B2)** is the recommended production configuration: it achieves the highest answer quality (ROUGE-L 0.534, faithfulness 0.844) and, counterintuitively, lower latency than the no-RAG Gemma baseline (61 s vs. 121 s). If latency is the primary constraint, **llama3.2:3b with RAG (B1)** offers a reasonable quality-speed trade-off at 70 seconds average response time with only a modest quality reduction.

Chapter 6

Conclusion

This thesis has presented the design, implementation, and empirical evaluation of a locally deployed retrieval-augmented generation chatbot for the ELTE Faculty of Informatics. The system allows students and staff to ask natural-language questions about the curriculum, course prerequisites, exam rules, and Neptun registration procedures, and receive answers grounded in official faculty documents — without sending any data to an external server.

6.1 Summary of Contributions

The thesis has delivered the following concrete artefacts and findings.

Document collection and ingestion pipeline. A resumable breadth-first crawler collects HTML pages, PDF documents, and DOCX files from the faculty's public web presence. An incremental ingestion pipeline maintains a SHA-256 manifest so that only new or changed files need to be reprocessed on subsequent runs, reducing re-indexing cost to a fraction of a full rebuild. The resulting corpus of 482 source files is chunked into 6112 overlapping text segments and stored in a ChromaDB vector index.

Offline RAG pipeline. Each chunk is encoded into a 384-dimensional dense vector by the `all-MiniLM-L6-v2` sentence embedding model. At query time the top-3 most similar chunks are retrieved by cosine similarity and assembled into a structured prompt that instructs the language model to answer only from the

provided context. The entire pipeline runs on a standard laptop with 8 GB of RAM and no GPU.

Local language model serving. Two quantised local models — `llama3.2:3b` and `gemma3:4b` — are served by Ollama. No query, no retrieved document chunk, and no generated answer leaves the host machine at any point during inference, satisfying the privacy and offline-operation requirements that motivated the project.

Production-quality backend and frontend. A FastAPI backend exposes the RAG pipeline through a minimal REST API, logs every interaction to a local SQLite database, and serves a browser-based chat interface styled to match the faculty's visual identity. The frontend supports session history, model switching, session deletion, and per-answer source references.

Empirical evaluation. A 20-question benchmark — 15 in-scope faculty questions and 5 out-of-scope control questions — was run across all four configurations. The results establish that RAG is the decisive factor in answer quality: ROUGE-L increases by 3.7× for `llama3.2:3b` and by 6.9× for `gemma3:4b` when retrieval context is added. Source faithfulness nearly doubles in both cases. The best-performing configuration, `gemma3:4b` with RAG, achieves ROUGE-L 0.534 and faithfulness 0.844, and produces shorter, more focused answers that also reduce average response latency relative to the no-RAG Gemma baseline.

6.2 Reflection on Goals

The system meets the two primary goals stated in Chapter 1.

Privacy. All computation — embedding, retrieval, and generation — occurs on the local machine. There is no network dependency after the initial model download, and no user query reaches any external service. This satisfies the GDPR-motivated offline-operation requirement.

Domain grounding. Without RAG, neither evaluated model reliably answers ELTE-specific questions: they have no parametric knowledge of faculty prerequisite rules or Neptun registration procedures. With RAG, both models produce answers that are substantially more accurate and faithful to the source documents. The

system successfully bridges the gap between what the language models know in general and what the faculty documents say specifically.

The 30-second response latency target (NF-4) was not met: all four configurations exceed it on CPU-only hardware, with median times ranging from 47 to 121 seconds. This is a known consequence of running billion-parameter models on a laptop without GPU acceleration, and is noted as a deployment constraint rather than an architectural failure.

6.3 Limitations

Three limitations are worth restating clearly.

Retrieval ceiling. The embedding-based retrieval achieves a hit-rate of 0.667, meaning one in three in-scope questions is answered without the most relevant chunk. The missed questions concern topics whose source content appears in shallow navigation text that is not well-represented after chunking. Improving chunking strategy or adding a re-ranking step is the highest-leverage improvement available.

Out-of-scope refusal. Only `llama3.2:3b` with RAG declines out-of-scope questions, and only 20% of the time. `gemma3:4b` does not refuse at all, falling back on parametric knowledge instead. A more robust refusal mechanism — such as a retrieval-score threshold below which the system returns a fixed "I don't know" response — is needed for a production deployment.

Benchmark scale. The 20-question test set is small and skewed toward prerequisite rules. A broader evaluation covering scholarships, exam appeals, credit transfer, and admissions would give a more complete picture of system performance.

6.4 Future Work

Several directions could extend the system beyond its current scope.

Improved retrieval. Replacing the fixed top-k retrieval with a cross-encoder reranker would improve chunk selection precision without changing the embedding index. Hybrid retrieval — combining dense vector search with a sparse BM25 index — is a well-established technique for handling exact-match queries (course codes, regulation numbers) that embedding models handle less reliably than semantic queries.

Retrieval-score threshold for refusal. A simple and robust out-of-scope detector would compare the cosine similarity of the top retrieved chunk against a calibrated threshold. If no chunk clears the threshold the system returns a fixed refusal response rather than forwarding a contentless prompt to the language model. This would eliminate the model-dependent refusal inconsistency observed in the evaluation.

Larger model on shared hardware. The latency constraint is a hardware constraint, not a software one. If the system were deployed on a faculty server with a mid-range GPU (e.g. NVIDIA RTX 3060, 12 GB VRAM), a 7B-parameter model could run at sub-10-second latency, bringing the system well within the original 30-second target while further improving answer quality.

Multilingual support. The faculty serves students who write questions in Hungarian as well as English. Replacing `all-MiniLM-L6-v2` with a multilingual sentence encoder (e.g. `paraphrase-multilingual-MiniLM-L12-v2`) and a multilingual instruction-tuned model would extend the system to Hungarian-language queries without re-indexing the corpus.

User feedback loop. The SQLite interaction log provides a natural foundation for a feedback mechanism: users could mark an answer as helpful or unhelpful. Aggregated feedback could be used to identify retrieval failures, surface low-quality chunks for manual review, and prioritise document updates in the corpus.

Bibliography

[1] Tom B. Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901.

[2] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.

[3] Georgi Gerganov et al. *llama.cpp: CPU-based Inference for LLaMA Models*. `https://github.com/ggerganov/llama.cpp`. Accessed: 2026-04-20. 2023.

[4] Tim Dettmers et al. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems* 36 (2023).

[5] EdTech Magazine. *How Three Universities Developed Their Chatbots*. `https://edtechmagazine.com/higher/article/2025/05/how-three-universities-developed-their-chatbots`. Accessed: 2026-04-20. 2025.

[6] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019, pp. 3982–3992.

[7] Yury A. Malkov and Dmitry A. Yashunin. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), pp. 824–836. DOI: `10.1109/TPAMI.2018.2889473`.

[8] Chin-Yew Lin. "ROUGE: A Package for Automatic Evaluation of Summaries". In: *Text Summarization Branches Out*. Association for Computational Linguistics, 2004, pp. 74–81.

List of Figures

List of Tables

List of Algorithms

List of Codes